Adaptação de CI/CD/CT — Spambase

1. Problema e dados

O grupo adaptou o projeto para um filtro acadêmico que encaminha mensagens suspeitas para spam a partir de atributos numéricos, para usuários/equipes de email. Fonte: UCI Spambase, Hopkins, Reeber, Forman e Suermondt (1999), DOI 10.24432/C53G6X, licença CC BY 4.0. Não incluímos textos ou endereços dos emails. São 4.601 registros e 57 atributos numéricos; após excluir seis linhas conflitantes e 391 repetições, restam 4.204. Divisão estratificada, semente 42: treino 2.522, holdout 841, reserva 841, sem sobreposição. O lote de CT usa 400 linhas da reserva, com origem registrada.

2. Contrato e resultado

Classe positiva: **spam (1)**. O falso positivo é mais caro: oculta uma comunicação legítima. Exigimos **precisão >= 0,93**, métrica principal, e **recall >= 0,85**, evitando detectar apenas poucos spams. O candidato não pode perder precisão para o campeão no mesmo conjunto. O campeão inicial atingiu precisão **94,70%** e recall **85,12%** no holdout; houve 16 falsos positivos e 50 falsos negativos. Modelo: log1p, StandardScaler e regressão logística.

3. Alterações por arquivo, seguindo a Parte C

Módulos abaixo pertencem a `src/mlops_demo/`; o nome do pacote foi preservado.

Etapa	Arquivos e adaptação
1	`data/raw/base_treino.csv`, `base_holdout.csv`: novas bases; `lotes/`: lote e proveniência JSON.
2	`spambase.py`, `data.py`: obtenção com SHA-256, limpeza e divisão reproduzível; remoção do dataset antigo.
3	`config.py`: 57 atributos, alvo spam, classe 1, colunas de drift e novos limiares.
4	`spambase.py`, `data.py`: esquema, tipos, finitude, percentuais, contagens e ausência de vazamento.
5	`features.py`, `train.py`, `params.yaml`: log1p, escala, regressão e seleção interna em cinco folds.
6	`evaluate.py`: precisão/recall de spam e matriz de confusão com classe positiva 1.
7	`data.py`, `monitor.py`: seis frequências simuladas; KS com p < 0,01; gatilho de quatro atributos.
8	`api.py`, `demo/smoke_test.py`: 57 entradas, esquema público, respostas spam/legítimo e HTTP 422.
9	`tests/`: quatro camadas adaptadas, novos testes específicos em cada camada; 75 testes na main.
10	`demo/cenarios.py`: cinco cenários; ensaios em cópias temporárias em `test_cenarios.py`.
11	`models/champion/`: campeão inicial, métricas, validação e hashes; promoção condicionada ao portão.
12	`README.md`, este relatório e `docs/`: domínio, execução, evidências, limites e uso de IA.

Infraestrutura: `gate.py` vincula a aprovação aos hashes e confirma preservação do campeão; `dvc.yaml`/`dvc.lock` registram treino; `constraints.txt` fixa dependências diretas; `Dockerfile` e os três workflows validam, retreinam e publicam a mesma imagem testada. Promoção exige habilitação explícita e permaneceu desativada para respeitar o orçamento de seis commits.

4. Evidências verificadas

Evidência	Execução do Actions e resultado
E1	CI na main: três jobs verdes, incluindo Docker.
E2	CI com falha controlada: testes de modelo vermelhos em Python 3.11/3.12.
E3	CT com drift aprovado: cinco atributos com drift; precisão 94,44%, recall 86,44%.
E4	CT reprovado no portão: recall 100%, precisão 46,88%; campeão preservado por SHA-256.
E5	CD com tag v1.0.0: CI, construção, teste, publicação e novo teste após download.

PR do lote: integrado por fast-forward. PR da falha: fechado sem merge. Pacote público e download anônimo com teste HTTP aprovados. Imagem: `ghcr.io/mainardikk/projeto_mlops:1.0.0`. A tag aponta ao commit operacional `7041842`; a última entrega acrescenta documentação. O candidato aprovado ficou como artifact; a imagem serve o campeão inicial. O desafio de promoção automática não foi realizado.

5. Limites, riscos e monitoramento

O portão verifica métricas em uma amostra finita, não garante generalização ou ausência de viés. O corpus é de 1999; o drift foi calibrado artificialmente, preservando rótulos, e o KS não tem correção de testes múltiplos. O recall inicial ficou perto do limite. Após uma primeira reprovação, o critério de seleção foi revisto: o holdout já havia sido consultado e não é um teste final independente. O histórico está em docs/VALIDACAO.md. Monitoraríamos precisão, recall e falsos positivos por segmento, prevalência, qualidade dos dados/rótulos, drift, latência e erros da API, com avaliação externa em dados recentes.

6. Contribuições e declaração de IA

O projeto é uma entrega do grupo, organizada em seis etapas com a seguinte distribuição de responsabilidades:

Integrante / GitHub	Commit / entrega	Responsabilidade
Thales Serschon (`thalescr77`)	1	Ambiente, importação do projeto e CI inicial.
Bruno Mainardi (`mainardikk`)	2	Fonte/licença, limpeza e divisão reproduzível dos dados.
Jhonatan Margraf (`Jhonatan-Margraf`)	3	Contrato, modelo, métricas, API e campeão inicial.
Davi Specia (`davispecia`)	4	Testes, drift, cenários, pipelines e publicação Docker.
Kaique Geska (`kaiquegeska`)	5	Falha controlada, E2/E4 e preservação do campeão.
Victor Ibarrola (`VictorOliveiraIbarrola`)	6	Relatório, README, evidências e conferência da entrega.

A divisão por etapa organiza as responsabilidades dentro do trabalho coletivo do grupo.

Uso de IA: o grupo utilizou OpenAI Codex no planejamento, scripts de dados, código de ML/API, testes, workflows, diagnóstico, documentação e operações Git/GitHub. As evidências foram verificadas por comandos e execuções reais do Actions.